FEASIBILITY STUDY

Running the PowerScale Admin Assistant on the cluster

From a constrained-CPU prototype to projected on-node performance

Question: the prototype is slow on a CPU laptop. Would it be usable running on a real PowerScale node, which is also CPU-based?

Answer: yes — the laptop's latency is an artifact of consumer hardware; a PowerScale Xeon node is a different AI-compute class. This document measures the prototype baseline, projects the node, and gives a one-command test to confirm it on real hardware.

1. The prototype's CPU latency is a floor, not the ceiling

The prototype runs on a consumer laptop CPU with only AVX2 — the weakest AI-compute tier. Measured on that machine with the shipped 3B model on a realistic retrieval prompt (~2,200 prompt + ~300 answer tokens):

Prototype CPU	Instruction tier	Prefill	Answer (typical)
Intel Core i5-13450HX (laptop)	AVX2 only	~54 tok/s	~106 s

Thread count barely moved it (6 vs 14 threads ≈ identical): the laptop is memory-bandwidth-bound with two memory channels and only performance-core headroom. This is the slowest hardware the assistant will ever run on.

Key point: the number to judge feasibility on is not this one. A PowerScale node changes three things at once — instruction set, core count, and memory bandwidth.

2. A PowerScale node is a different compute class

PowerScale all-flash and hybrid nodes run Intel Xeon Scalable processors, which carry the AI-acceleration instructions the laptop lacks, across many more cores and memory channels.

Attribute	Prototype laptop	PowerScale node (Xeon Scalable)
AI instructions	AVX2	AVX-512 + VNNI (Ice Lake); AMX on newest gen
Cores (usable)	~6 effective	12 – 64 (e.g. F900 = dual 32-core)
Memory channels	2	8 per socket (much higher bandwidth)
RAM	tens of GB	hundreds of GB
GPU	none	none (all CPU) — same as prototype

VNNI accelerates the int8 quantized matrix math that dominates prefill; AMX (on Sapphire-Rapids-class nodes) is a dedicated matrix engine that pushes CPU inference toward GPU-class throughput. None of this is new work — it is the platform the assistant would already be running on.

Implication for packaging: the prototype pins an old AVX2 build of the runtime to avoid a crash on consumer chips. On a node, that pin is dropped — ship the AVX-512/AMX build matched to the node's generation.

3. Projected performance on the node

Scaling the measured prototype baseline by the instruction-set and core/bandwidth gains, under a conservative on-cluster thread cap (see §4). Ranges, not point estimates — to be replaced by the measured number from §5.

Node class	AI tier	Speed-up vs prototype	Projected answer
F200 / F600 (Ice Lake)	AVX-512 + VNNI	~3–4×	~27–40 s
F900 (dual 32-core)	AVX-512 + VNNI	~4–6×	~18–27 s
Sapphire-Rapids class	AVX-512 + AMX	~8–15×	~7–14 s

Assumptions. Baseline ~106 s (measured). VNNI ≈ 2–4× prefill over AVX2; AMX ≈ 5–10×; plus real core/bandwidth scaling beyond the laptop's plateau, capped by the governor in §4. Prefill dominates, so instruction set matters most. Streaming means first words appear well before the totals above.

Read: on current-gen nodes an answer lands in tens of seconds; on AMX-class nodes, in roughly ten. For a documentation assistant that saves a support case, that is usable.

4. Storage stays first — the governor bounds the projection

A node's cores are paid for by the customer to serve storage, not to run a chatbot. The assistant runs as a low-priority, thread-capped service so client I/O and cache always win. That cap is a deliberate trade: it gives up peak inference speed to protect the storage SLA.

- The projections above assume a capped allocation (a slice of cores), not the whole node — so they are conservative; a dedicated box would be faster.
- The assistant is bursty (it works only during a question), so its cap is idle most of the time.
- N_THREADS in the prototype is the seed of this governor; on a node it is pinned to a fixed slice.

5. How we confirm it — one command on real hardware

Projections become a number by running the bundled bench on hardware that matches the node generation (a lab node, or an Ice-Lake / Sapphire-Rapids cloud VM):

```
python bench/feasibility_bench.py
```

It detects the CPU's AI tier (AVX-512 / VNNI / AMX), times prefill and generation at several thread counts, and prints the projected answer latency — the same table as §3, measured. Acceptance gate before committing to on-node deployment:

- **Answer latency at the capped thread count meets the target for that node class (e.g. ≤ 15 s on F900-class).**
- **With the assistant active under load, client I/O throughput and P99 latency stay within ~2% of baseline.**

If a class misses the gate, the fallback is the InsightIQ VM (off-node compute) — already the recommended first deployment.

6. Risks & unknowns

- Memory-bandwidth ceiling — past a point, more threads stop helping; the bench measures where that is per node.
- Contention with storage services under peak load; the governor mitigates but must be validated at P99.
- Node-generation variance — older archive (A-series) nodes are a weaker tier; feature them only where the node clears the gate.
- Answer quality is fixed to the 3B model; a larger model would be higher quality but slower — a separate trade to test on-node where RAM is ample.

7. Recommendation

Feasible, pending one measurement. The prototype proves the software; the node provides the missing compute. Validate on a matching Xeon VM with the bench, ship the AVX-512/AMX runtime build for the node's generation (not the prototype's AVX2 pin), and keep the InsightIQ-VM path as the low-risk first step while on-node clears its gate.

Projections are engineering estimates pending the on-node bench in §5. Prototype baseline measured on an Intel i5-13450HX (AVX2), 3B model, ~2,200+300-token prompt.